

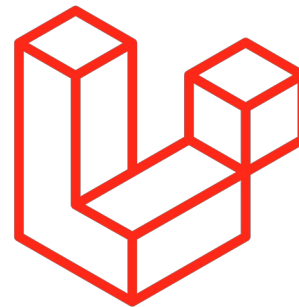
**INSTITUTO FEDERAL**  
Sul-rio-grandense

Câmpus  
Pelotas



# INTRODUÇÃO AO LARAVEL

Instalação, Estrutura, Arquitetura MVC e Rotas



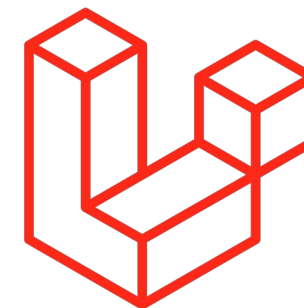
CSTSI - V SEMESTRE  
DESENVOLVIMENTO BACK-END 2  
Prof. Gill Velleda Gonzales | [gillgonzales@ifsul.edu.br](mailto:gillgonzales@ifsul.edu.br)  
Setembro, 2025

# Introdução ao Laravel

Tópicos da aula



- Framework - Fundamentos
- Instalação
- Estrutura
- MVC
  - Rotas
  - Model
  - View
  - Controller
  - Exemplo
- Atividade Prática



*php* 8.3



docker



MariaDB

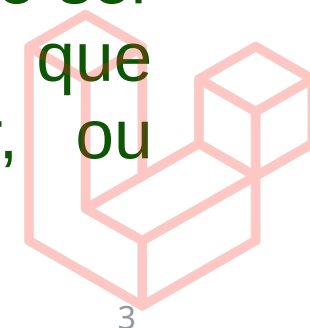


- O que é um Framework?

Um Framework é literalmente uma “**Janela de Trabalho**”, ou seja, um espaço pré-definido onde iremos construir algo reaproveitando soluções já implementadas.

- Framework vs Biblioteca (Libs)

Um **Framework** utiliza várias **bibliotecas** prontas para problemas em que já existem soluções conhecidas, testadas e validadas. A diferença é que o **Framework** possui uma **forma de trabalhar** pré-definida, que deve ser considerada para que tudo funcione perfeitamente. Enquanto que **bibliotecas** resolvem pequenos problemas sem se preocupar, ou determinar, como o todo deve funcionar.

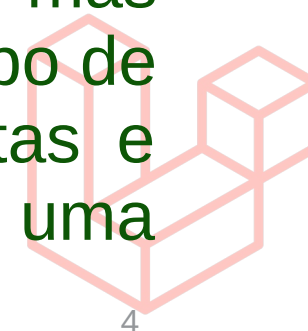




- **Analogia:**

Pense em um **Framework** como uma caixa de ferramentas de um eletricista. Nesta **caixa de ferramentas** existem todas as variadas **ferramentas** que esse profissional precisa para um determinado trabalho com um **objetivo específico**, manutenção elétrica. Agora imaginamos a caixa de ferramentas de um mecânico, igualmente existem várias **ferramentas** para agilizar o serviço de manutenção mecânica, automotiva. Pense também na caixa de ferramentas de um encanador.

Todas as caixas possuem ferramentas (**Bibliotecas**) em comum, mas possuem **objetivos** e “**formas de trabalhar**” diferentes para cada tipo de serviço. Provavelmente encontraremos nas duas caixas ferramentas e objetos em comum como chave de fenda, alicates variados, uma furadeira e etc.



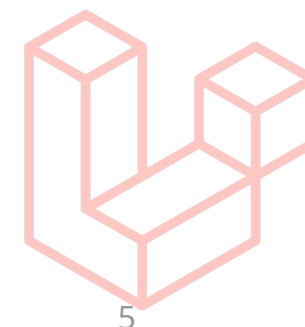


- **Por que um Framework:**

Não invente a roda! Simples! Um **Framework** traz agilidade ao processo de desenvolvimento.

- **Existem vários frameworks?**

Sim! Por que nem todos eletricitas, mecânicos ou programadores trabalham do mesmo jeito, então cada um prefere um caixa de ferramenta que melhor atenda ao seu trabalho.



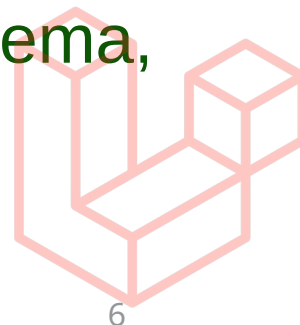
# Framework - Fundamentos

## Primeiros conceitos



- **E qual escolher? Depende de :**
  - Requisitos do projeto
  - Prazo de entrega / curva de aprendizagem
  - Documentação / comunidade
  - Objetivos profissionais (carreira)
  - Preferencias pessoais
  - Etc...

Não existe uma ferramenta, ou melhor, uma caixa de ferramentas que resolva todos os problemas igualmente em eficiência e qualidade. Temos que aprender a mudar a caixa de ferramentas de acordo com o problema, isso serve também para linguagens de programação.



# Instalação do Framework

Instalando o Framework Laravel

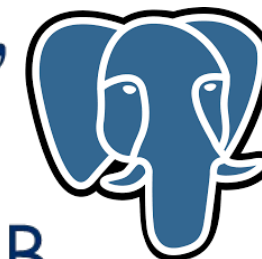


- **Requisitos do Laravel 11.x**

- Para utilizar o **framework** serão necessárias as seguintes ferramentas:

- PHP  $\geq 8.1$
- Composer
- Node e npm (Vite)
- Banco de Dados Suportado
- Ou configurar tudo através de containers Docker:
  - Docker Desktop: Windows ( WSL2) e Mac
  - Docker Composer: Linux

php 8.3





# Instalação do Framework

Instalando o Framework Laravel via Container

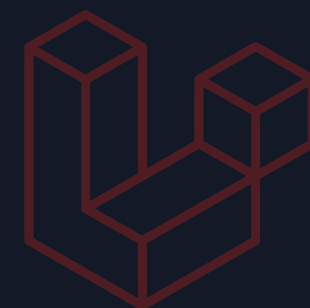


- Criando um projeto do Laravel com Docker

Exemplo para Linux do projeto *sail-app-mysql* com serviço do *mysql* definido na *query with* :

```
$> curl -s "https://laravel.build/sail-app-mysql?with=mysql" | bash
```

```
echoes@echoes-Inspiron-15-3567:/media/Dados/Projetos/prototipos$ curl -s "https://laravel.build/sail-app-mysql?with=mysql" | bash
latest: Pulling from laravelsail/php82-composer
bb263680fed1: Pull complete
0825793cba86: Pull complete
de3c011d207b: Pull complete
7e3c5bd9650e: Pull complete
e5ef4f07283b: Pull complete
729644263cfa: Pull complete
9c646fe3f3c5: Pull complete
e7c9e0a2bc78: Pull complete
87fd6a1f6441: Pull complete
136b5bfaa68e: Pull complete
cf33f30cc64a: Pull complete
9250c2dfc674: Pull complete
4f4fb700ef54: Pull complete
59bc22386503: Pull complete
376a1c60a489: Pull complete
Digest: sha256:37549f980be6146efcb16b0dc352644edfc5cca32b9862494631eb197ef89737
Status: Downloaded newer image for laravelsail/php82-composer:latest
```





# Instalação do Framework

Instalando o Framework Laravel via Container



## Criando um projeto do Laravel com Docker

- O **build** de criação do projeto através de **containers** difere de acordo com o **OS**.
- O comando da imagem anterior criará o projeto com o nome **sail-app-mysql**.

```
=> [11/11] RUN chmod +x /usr/local/bin/start-container
                                0.8s
=> exporting to image
                                11.6s
=> => exporting layers
                                11.6s
=> => writing image sha256:0c182a0d9ef6bc379a1e13b32c0386fb0062b887d1a9b943d576e73a93026282
                                0.0s
=> => naming to sail-8.2/app
                                0.0s
```

Please provide your password so we can make some final adjustments to your application's permissions.

[sudo] password for echoes:

Thank you! We hope you build something incredible. Dive in with: `cd sail-app-mysql && ./vendor/bin/sail up`

`echoes@echoes-Inspiron-15-3567:/media/Dados/Projetos/prototipos$ ls`

`cstsi_daoo_php_exemplos_poo` `forum` `laravel` `php-project` `sail-app-mysql` `threejs_projects`  
`tsi_daoo`

# Instalação do Framework

Instalando o Framework Laravel via Composer



- Criando um projeto do Laravel sem Containers

Possuindo o gerenciador de dependências do *php* (*composer*) instalado, podemos instalar a versão atualizada do *framework* Laravel!

- Execute o seguinte comando:

```
$> composer create-project laravel/laravel nomeDoProjeto
```



- O *composer* irá criar uma pasta com o nome **nomeDoProjeto** com os arquivos do *framework*.
- Para mais detalhes sobre a instalação do **Laravel**, visite a documentação no seguinte endereço:
- <https://laravel.com/docs/12.x#your-first-laravel-project>
- Outra forma de instalar um projeto do Laravel é através de seu **instalador global**:

```
$> composer global require laravel/installer
```

# Instalação do Framework

Instalando o Framework Laravel via Composer



- **Instalador Global do Laravel**

Com o Laravel-Installer instalado no *composer* é possível criar um projeto novo com o simples comando:

```
$> laravel new example-app
```

- No entanto é preciso configurar o ambiente adicionando a pasta do **composer** no **PATH** do sistema.
- No **Linux** adicionamos no arquivo **.profile** do usuário, na pasta raiz do usuário (~):

```
23
24 # set PATH so it includes user's private bin if it exists
25 if [ -d "$HOME/.local/bin" ] ; then
26     PATH="$HOME/.local/bin:$PATH"
27 fi
28 export PATH="$PATH:$HOME/.config/composer/vendor/bin"
```

- **export PATH="\$PATH:\$HOME/.config/composer/vendor/bin"**



# Instalação do Framework

Instalando o Framework Laravel via Composer



- **Instalador Global do Laravel**

Para instalar o **Laravel-Installer** no Windows é necessário editara variável PATH do windows da seguinte forma:

- Pesquise “**Variáveis de Ambiente**”
- Ou vá nas propriedades do sistema e procure “**Variáveis de Ambiente**”
- Em **Variáveis do Sitema**:
  - Edite o conteúdo da variável **Path**, adicionando ao final:
    - **;%USERPROFILE%\AppData\Roaming\Composer\vendor\bin;**
    - Basicamente é o caminho da pasta **bin** do composer.
- **Documentação Oficial !**
- **Tutorial Windows 10 (Fazer o tutorial)**



# Instalação do Framework

Instalando o Framework Laravel via Composer



Portanto, podemos criar o nosso projeto de exemplo chamado ***aula\_03*** !

Execute o seguinte comando:

```
$> composer create-project laravel/laravel aula_03
```

Ou com ***laravel-installer***:

```
$> laravel new aula_03
```

```
INFO Application ready in [aula_03]. You can start your local development using:
```

```
→ cd aula_03
```

```
→ php artisan serve
```

```
New to Laravel? Check out our bootcamp and documentation. Build something amazing!
```

```
~/Projetos/prototipos/laravel
```

**Laravel instalado**, dentro do diretório do projeto usaremos muito o ***CLI Artisan***, como por exemplo, para iniciar o servidor de desenvolvimento:

```
$> php artisan serve
```

# Instalação do Framework

Instalando o Framework Laravel via Composer



- **Instalador Global do Laravel**

Com o Laravel-Installer instalado no *composer* é possível criar um projeto novo com o simples comando:

```
$> laravel new example-app
```

- Neste caso existe uma interface que auxilia na criação do projeto.

```
~/Projetos/prototipos/laravel
> laravel new aula_03

Laravel

Would you like to install a starter kit?
> ● No starter kit
  ○ Laravel Breeze
  ○ Laravel Jetstream
```



# Instalação do Framework

Instalando o Framework Laravel via Composer



- **Instalador Global do Laravel**

Uma das configurações solicitadas é o banco de dados, solicitando um dos banco de dados suportados. O instalador ainda avisa aqueles em que não estão instaladas as dependências corretas do php.

```
Which database will your application use? _____
```

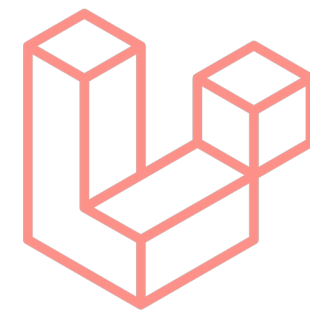
```
> ● SQLite
```

```
○ MySQL
```

```
○ MariaDB
```

```
○ PostgreSQL (Missing PDO extension)
```

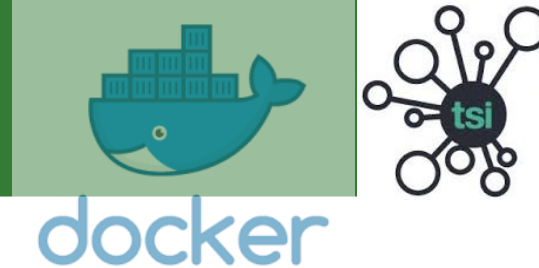
```
○ SQL Server (Missing PDO extension)
```



- Neste caso eu deveria instalar o php8.4-pgsql (sudo apt install...).
- **SQLite** é um banco de dados baseado em um arquivo, interessante para ambiente de desenvolvimento.

# Instalação do Framework

Utilizando o Docker com o Laravel Sail



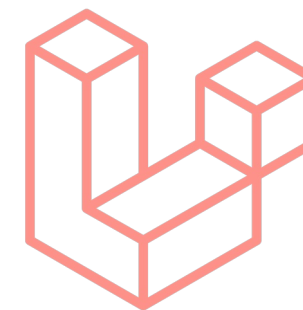
- **Instalação do Laravel Sail como Dependência Dev**

Com o Laravel Sail instalado via *composer* é muito mais fácil gerenciar o container Docker da aplicação. Use o comando a seguir para instalar o **Laravel Sail**:

```
$> composer require laravel/sail --dev
```

- Após a instalação do sail, na pasta do projeto, executamos o comando do **artisan** para preparar o projeto como um **container docker (containerização)**.
- ```
$> php artisan sail:install
```
- Este comando cria os arquivos necessários para a configuração do docker, bem como ajuda na escolha dos serviços ( banco de dados)

- **Documentação Laravel Sail**

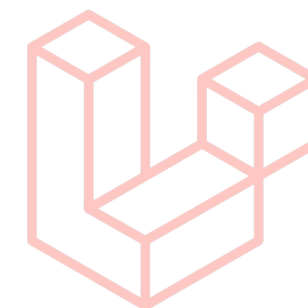


# Instalação do Framework

Utilizando o Docker com o Laravel Sail



- **Configuração do Laravel com Docker**
- Após a preparação com o sail, o projeto agora possui o arquivo ***compose.yaml***
- Este arquivo descreve a configuração dos ***containers*** a serem utilizados para o ***laravel*** e os demais serviços escolhidos, neste caso apenas o ***mariadb***.
- Não existe serviço para o gerenciador ***phpmyadmin***.
- No entanto podemos baixar um *container* e configurá-lo para conectar ao ***container*** do ***laravel***.
  - ***Comando para Linux !***
- Mas podemos também modificar o arquivo ***compose.yaml***.



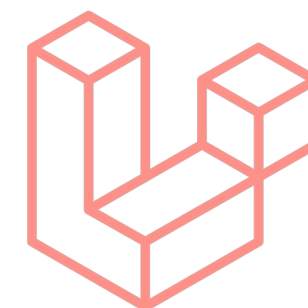
# Instalação do Framework

Instalando o Framework Laravel via Container



- Adicionando o serviço do phpmyadmin ao container do laravel
- Adicionando o phpmyadmin ao ***compose.yaml***

```
docker-compose.yml x
docker-compose.yml
49         timeout: 5s
50     phpmyadmin:
51         image: 'phpmyadmin/phpmyadmin:latest'
52         ports:
53         - '8091:80'
54         environment:
55         - PMA_HOST: 'mariadb'
56         networks:
57         - sail
58         depends_on:
59         - mariadb
60     networks:
```



- Arquivo de exemplo do ***compose.yaml*** 🌐!

# Instalação do Framework

Instalando o Framework Laravel via Container



- Criando um projeto do Laravel com Container Docker
- Para inicializar o *container* do *Laravel* usamos os comandos do SAIL:

```
$> ./vendor/bin/sail up
```

- Para parar os *containers*: use o comando **down**

```
$> ./vendor/bin/sail down
```

- Podemos ainda criar um alias no sistema:

```
$> alias sail='[ -f sail ] && sh sail || sh vendor/bin/sail'
```

- O alias deverá ser colocado no **.zshrc** ou **.bash** para carregar na inicialização do terminal.

```
$> sail up
```

- Para mais detalhes sobre a instalação utilização do **Laravel** através de **containers**, veja a documentação do **Laravel Sail**



# Instalação do Framework

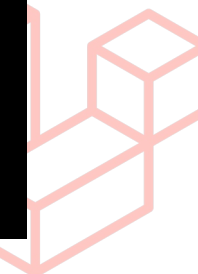
Instalando o Framework Laravel via Container



- **Construindo o container Docker**
- Após toda a configuração de serviços e portas, rodamos o comando `sail up`, e na primeira execução, o container será construído (build).

```
$> sail up
```

```
[+] Building 22.5s (8/18)                                docker:desktop-linux
=> => transferring context: 1.12kB                        0.0
=> [laravel.test 2/14] WORKDIR /var/www/html              1.5
=> [laravel.test 3/14] RUN ln -snf /usr/share/zoneinfo/UTC /etc/localtime && echo UTC > /etc/timezone 0.7
=> [laravel.test 4/14] RUN echo "Acquire::http::Pipeline-Depth 0;" > /etc/apt/apt.conf.d/99custom && 0.6
=> [laravel.test 5/14] RUN apt-get update && apt-get upgrade -y && mkdir -p /etc/apt/keyrings & 17.0
=> => # Get:6 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
=> => # Get:7 http://archive.ubuntu.com/ubuntu noble/restricted amd64 Packages [117 kB]
=> => # Get:8 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [19.3 MB]
=> => # Get:9 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2356 kB]
=> => # Get:10 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1136 kB]
=> => # Get:11 http://archive.ubuntu.com/ubuntu noble/main amd64 Packages [1808 kB]
```





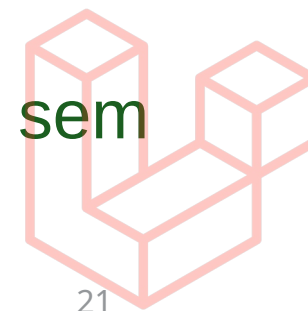
# Acesso ao Framework

Configurações para executar no servidor



- **Criação da estrutura padrão do banco de dados:**
  - O Laravel possui a sua própria estrutura pré-definida de tabelas e devemos criá-las no banco antes de iniciar o servidor através do ***artisan migrate***.
  - O ***artisan*** é uma interface **CLI** (**C**ommand **L**ine Interface) que facilita muito o desenvolvimento e a automatização de processos envolvendo o framework Laravel.
  - Executando com container Docker e utilizando o **SAIL**:  
**\$> sail artisan migrate:fresh**  
O SAIL executa o comando do artisan no container.
  - Executando diretamente o ***artisan*** com o comando ***migrate*** sem containers:

**\$> php artisan migrate:fresh**



# Acesso ao Framework

Configurações para executar no servidor



- **Como rodar o framework:**

- O Laravel possui o seu próprio comando para iniciar o servidor de desenvolvimento
- Executando com container Docker:

```
$> sail up
```

Onde a porta será definida nas configurações do serviço no arquivo ***compose.yml***

- Executando o ***artisan*** com o comando ***serve*** e a opção ***--port***:

```
$> php artisan serve --port=8082
```

O ***artisan*** é uma interface ***CLI*** (Command Line Interface) que facilita muito o desenvolvimento e a automatização de processos envolvendo o framework Laravel.



# Acesso ao Framework

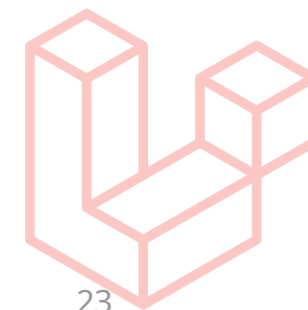
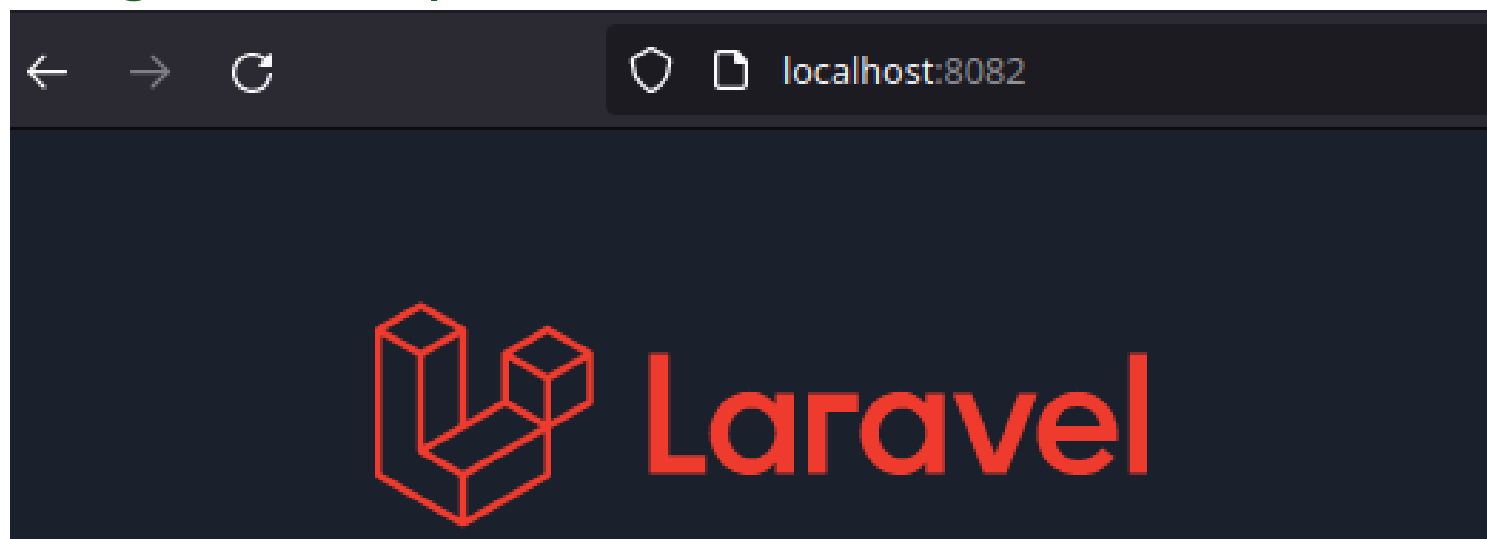
Configurações para executar no servidor



- **Acesso via Artisan: `php artisan serve --port=8082` (RECOMENDADO)**

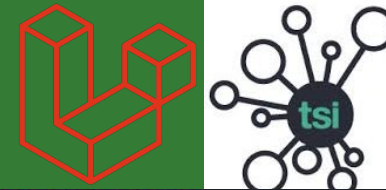
```
20:04:39 echoes@echoes-Inspiron-15-3567 ...Projetos/tsi_daoo/introLaravel <?PHP 7.4.3
$ php artisan serve --port=8082
Starting Laravel development server: http://127.0.0.1:8082
[Tue Feb 22 20:04:43 2022] PHP 7.4.3 Development Server (http://127.0.0.1:8082) started
[Tue Feb 22 20:04:51 2022] 127.0.0.1:37386 Accepted
[Tue Feb 22 20:04:51 2022] 127.0.0.1:37386 Closing
[Tue Feb 22 20:04:52 2022] 127.0.0.1:37390 Accepted
[Tue Feb 22 20:04:52 2022] 127.0.0.1:37390 [200]: GET /favicon.ico
[Tue Feb 22 20:04:52 2022] 127.0.0.1:37390 Closing
```

- **Acesso no navegador: `http://localhost:8082`**

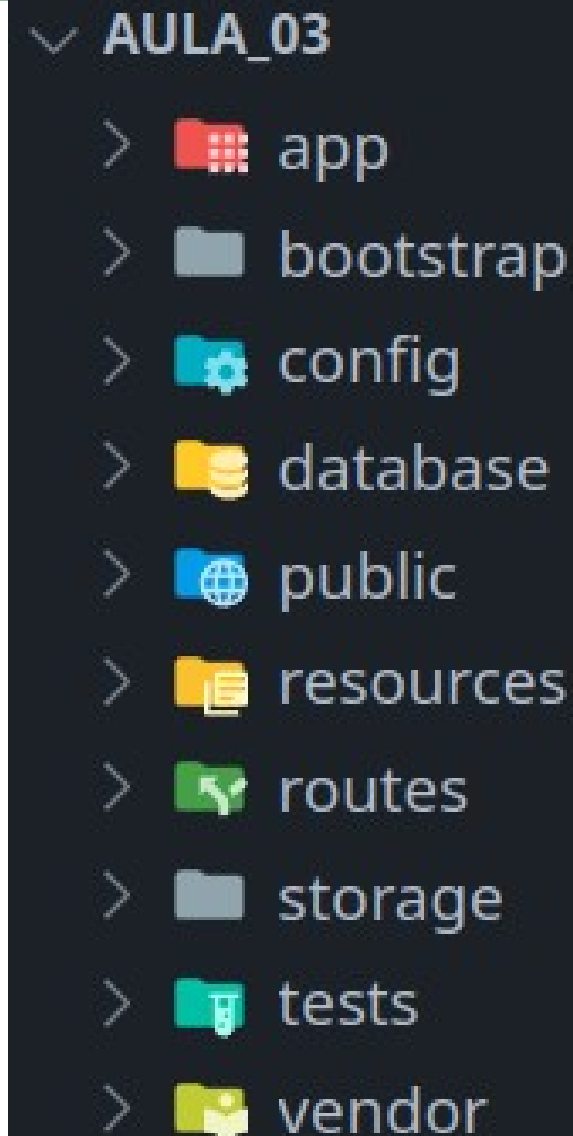


# Estrutura do Framework

Conhecendo o Framework Laravel – Diretórios e Arquivos



- **App:** Onde estará a maior parte da aplicação (Controller e Model)
- **Bootstrap:** Inicialização do framework
- **Config:** Arquivos de configuração do framework
- **Database:** Definições do banco de dados (*migrations, seeds*)
- **Lang:** Suporte a múltiplos idiomas de acordo com o *locale*.
- **Public:** Entrada web e arquivos públicos do servidor (templates html, arquivos de download, scripts compilados, estilos, etc...)
- **Resources:** Recursos para scripts (*views* [templates], *js*, *sass*, *less*, *css*, *imagens* etc..)
- **Routes:** Arquivos de definições de rotas (*web* e *api*).
- **Storage:** Diretório de armazenamento (*Upload*)
- **Tests:** Configuração de testes unitários (*Unit*)
- **Vendor:** Diretório com as dependências de terceiros, ignorado pelo versionamento (*.gitignore*).



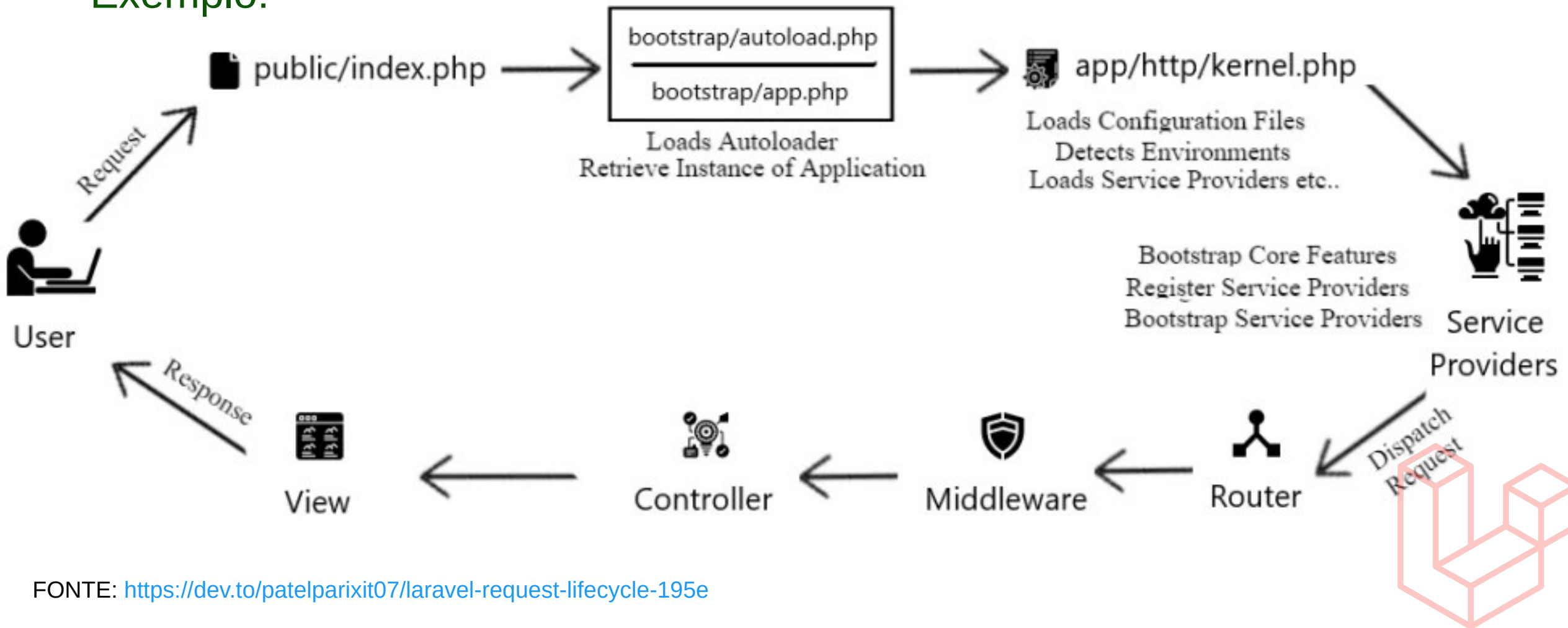
# Request Lifecycle do Laravel

Ciclo de Vida de Requisições no Laravel



- Request Lifecycle

- Exemplo:



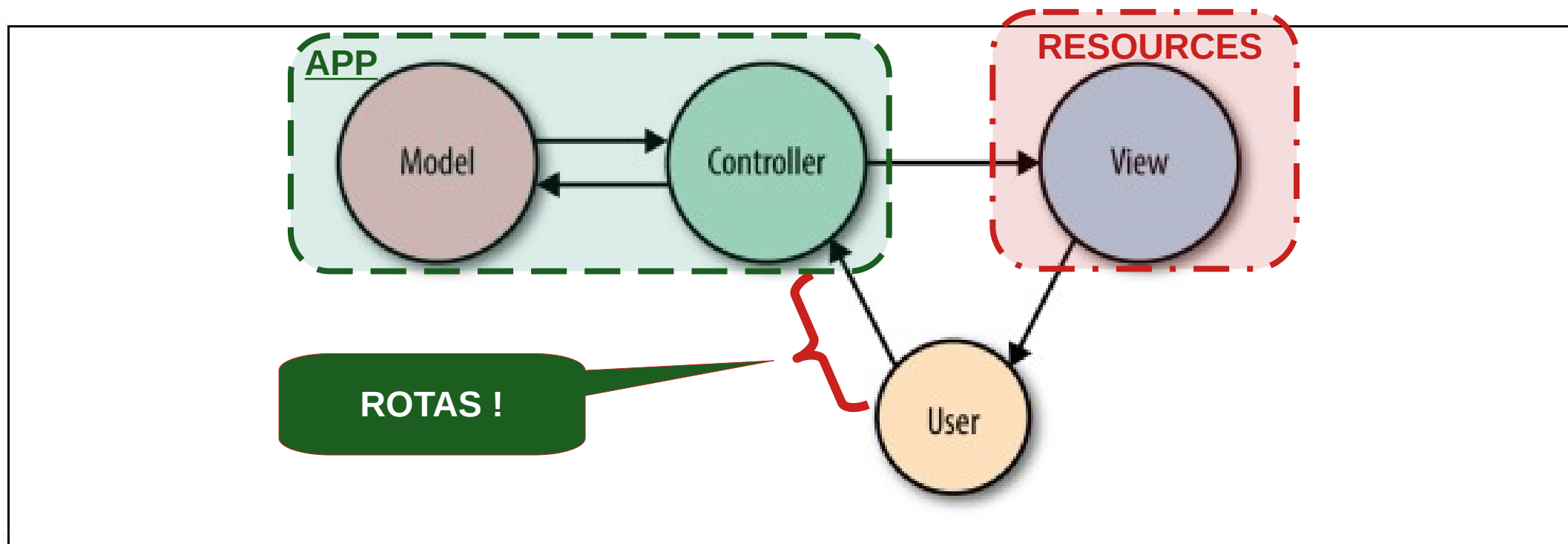
FONTE: <https://dev.to/patelparixit07/laravel-request-lifecycle-195e>

# MVC no Laravel

Padrão Model-View-Controller no Framework Laravel



- **Padrão Arquitetural:**



- No **Laravel** as camadas **Model** e **Controller** estão na pasta **App**.
- A camada de **View** é desenvolvida como templates na pasta **Resources**.
- **Controllers** ou **Views** são acessados via **ROTAS** (pasta **Routes**)



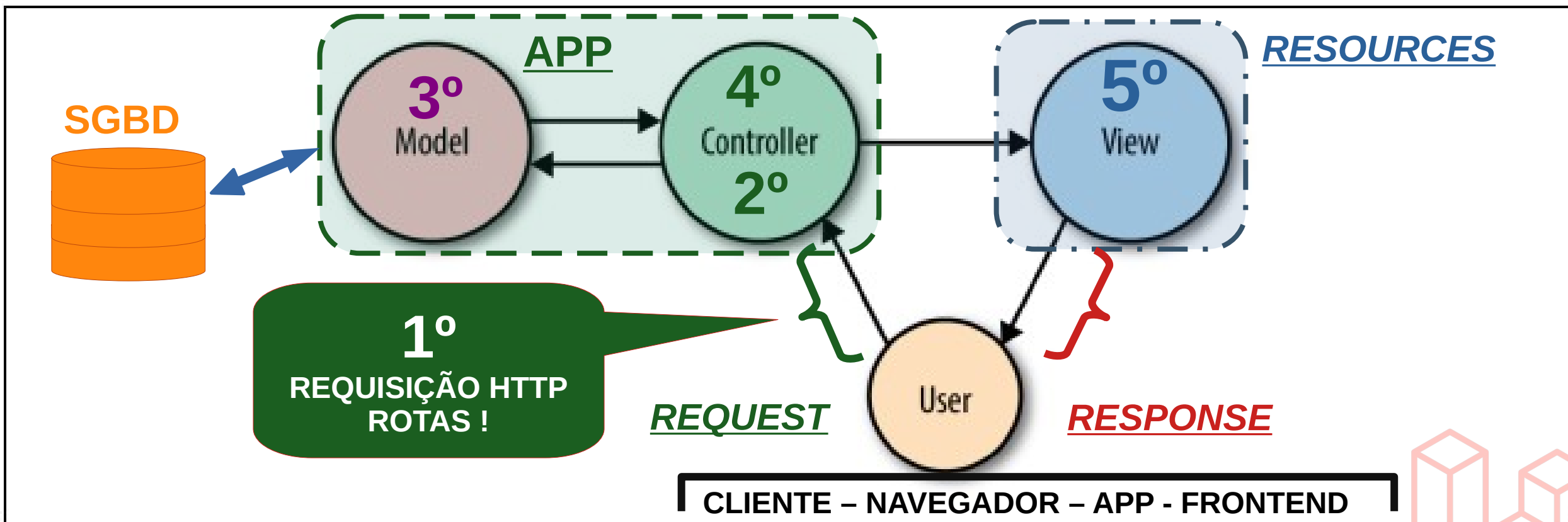


# MVC no Laravel

Padrão Model-View-Controller no Framework Laravel



- **Fluxo:**



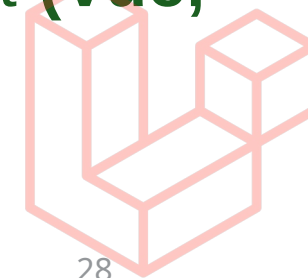
# MVC no Laravel

Padrão Model-View-Controller no Framework Laravel



- **Fluxo:**

- 1) **Rotas** → repassa a requisição para **Controller** ou redireciona para **View**
- 2) **Controller** → recebe a requisição e acessa os métodos da **Model** (**CRUD**)
- 3) **Model** → recebe as chamadas do **Controller** e retorna com os **dados**
- 4) **Controller** → recebe dados da camada de modelo e redireciona para a resposta, que pode ser a chamada de um template da **View**, ou um retorno de resposta **http (API REST)**.
- 5) **View** → recebe dados da camada de controle e renderiza a página através de algum motor de templates (**Blade**) ou framework javascript (**Vue, ReactJS, Angular etc...**)



# MVC no Laravel

Padrão Model-View-Controller no Framework Laravel

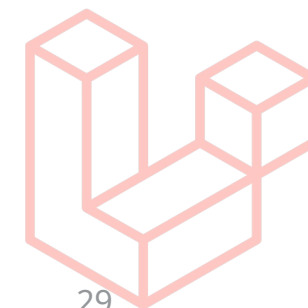
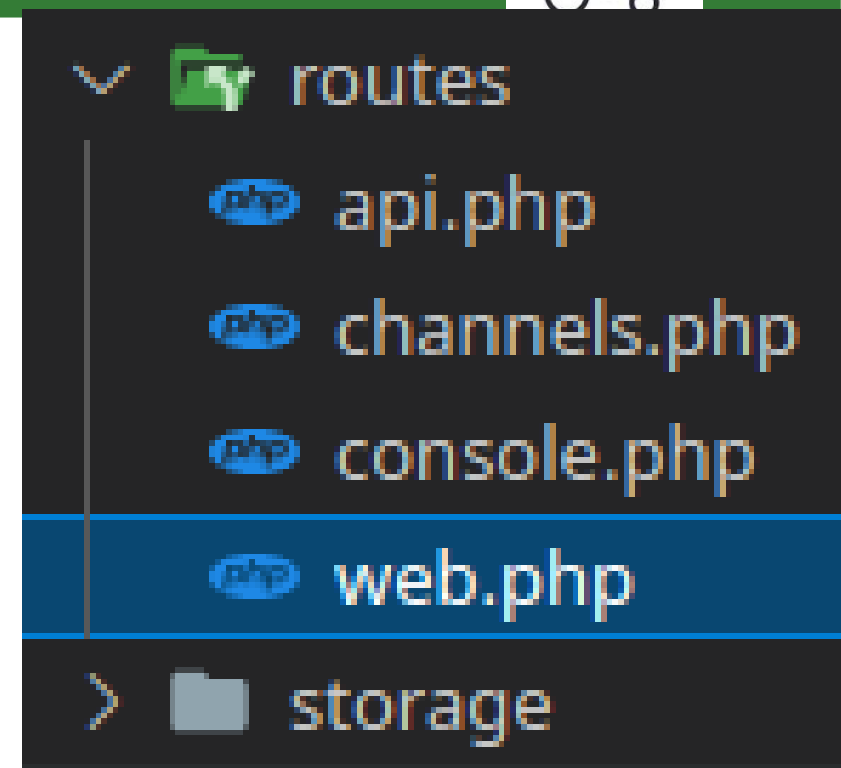


- **Rotas:**

- Determina as chamadas para os Controladores
- Gerencia e agrupa rotas por Controlador
- Mapeia verbos HTTP: ***get, post, put, delete, patch, options;***
- Pode redirecionar para a ***View*** diretamente chamando um template: ***view('nome\_template');***

-  **Route:**

- Agrupamento de rotas por controlador
- Rotas restritas que requerem autenticação
- Rotas para ***/api*** automaticamente pelo arquivo ***routes/api.php***



# MVC no Laravel

Padrão Model-View-Controller no Framework Laravel

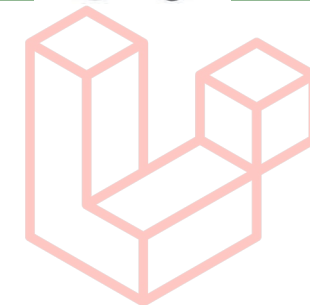


- **Rotas:** Exemplo típico de rotas para autenticação e restritas por tokens JWT.

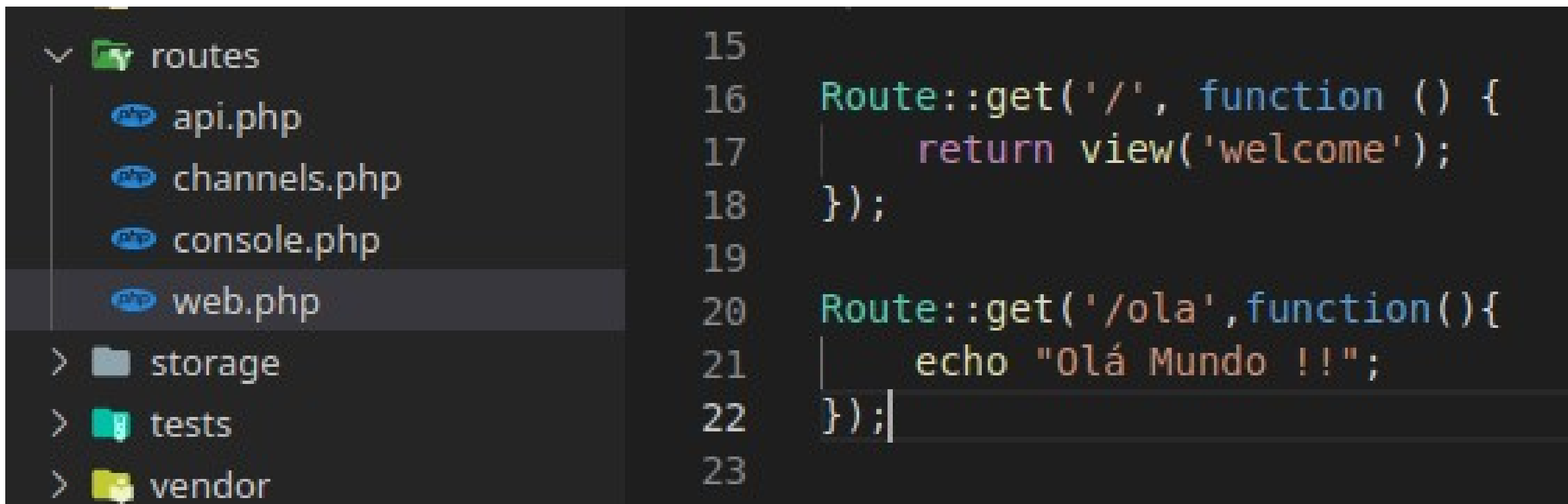
```
Route::prefix('v1')->group(function (){
    Route::post('/login',[LoginJwtController::class,'login'])->name('login');
    Route::get('/logout',[LoginJwtController::class,'logout'])->name('logout');
    Route::get('/refresh',[LoginJwtController::class,'refresh'])->name('refresh');
    Route::group(['middleware'=>['jwt.auth']], function(){
        Route::name('real_states.')->group(function (){
            // Route::get('/',[RealStateController::class,'index']); #/api/v1/real-states/
            // Route::get('/{real_state}',[RealStateController::class,'show']);
            // Route::post('/',[RealStateController::class,'save'])->middleware('auth.basic'); #/
            // Route::delete('/{real_state}',[RealStateController::class,'remove'])->middleware('auth');
            // Route::put('/{real_state}',[RealStateController::class,'update'])->middleware('auth');
            Route::apiResource('real-states', RealStateController::class);
        });
    });
});
```

# MVC no Laravel

Padrão Model-View-Controller no Framework Laravel



- **Rotas:** Exemplo simples no projeto *introLaravel*.
- Edite o arquivo *routes/web.php* para criar a rota “ola”
- Logo passe uma função anônima que imprime “**Olá Mundo !!!**”

A screenshot of a code editor with a dark theme. On the left, a file explorer shows the 'routes' directory expanded, with files 'api.php', 'channels.php', 'console.php', and 'web.php'. 'web.php' is selected. Below it are folders 'storage', 'tests', and 'vendor'. On the right, the code in 'web.php' is shown with line numbers 15 to 23. The code defines two routes: a root route '/' returning a 'welcome' view, and a route '/ola' that echoes 'Olá Mundo !!!'.

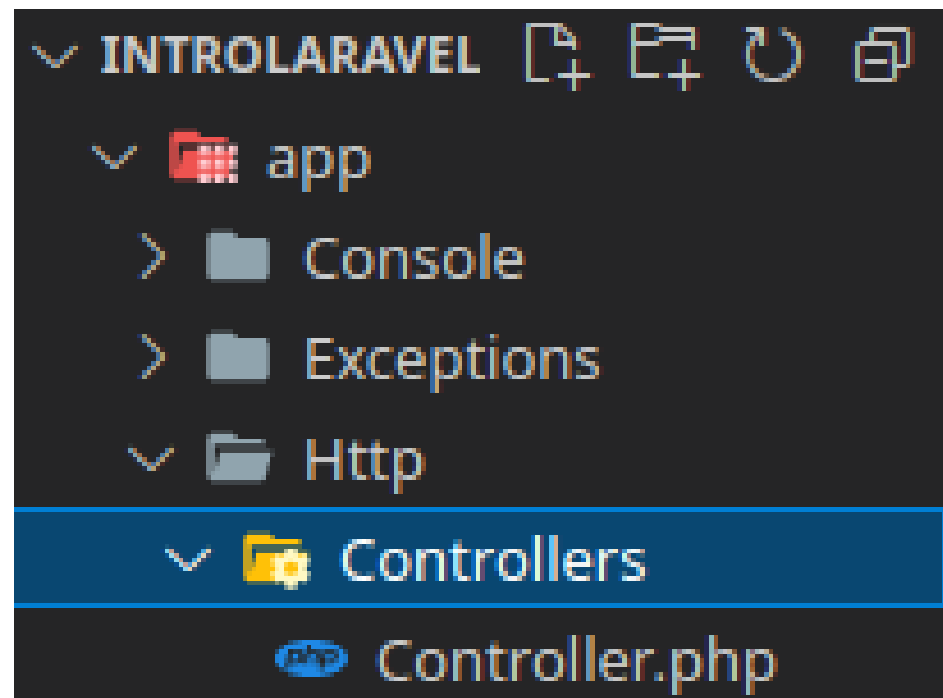
- Acesse a rota <http://localhost/ola>

# MVC no Laravel

Padrão Model-View-Controller no Framework Laravel



- **Controllers:**
- Localizados no diretório */app/Http/Controllers/*
- Estendem a classe **Controller** básica.
- São usados para intermediar o acesso à camada de modelo (**Model**)
- Também executam redirecionamentos e respostas **HTTP**
- É no **Controller** que estará as chamadas as camadas da aplicação:
  - Tratamento de parâmetros da **requisição** e **autorização**;
  - Acesso aos **Models** específicos para recuperação ou persistência de dados;
  - Redirecionamento de respostas para a requisição através da **View**;





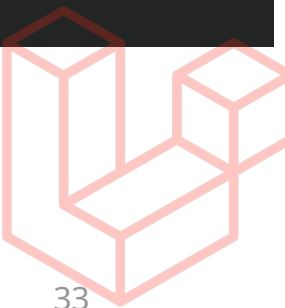
# MVC no Laravel : CONTROLLER

Padrão Model-View-Controller no Framework Laravel



- **Controllers:** Exemplo de um Controlador

```
12 class RealStateController extends Controller
13 {
14     private $realState;
15
16     public function __construct(RealState $realState){...}
17
27
28     public function index(){
29         $realState = auth(guard: 'api')->user()->real_state()->paginate(10);
30         return response()->json($realState, status: 200);
31     }
```



# MVC no Laravel : CONTROLLER

Padrão Model-View-Controller no Framework Laravel



- **Controllers:** Criando o primeiro Controlador do projeto *introLaravel*
- Vamos usar o comando *artisan*:

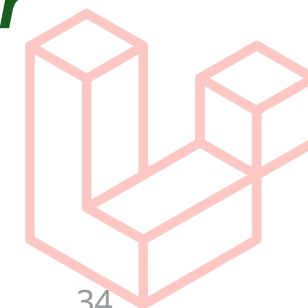
```
$> php artisan make:controller HomeController
```

```
echoes@echoes-Inspiron-15-3567 ...Projetos
```



```
$ php artisan make:controller HomeController  
Controller created successfully.
```

- O arquivo HomeController.php deverá ser criado na pasta **Controller**  
**/app/Http/Controllers/**HomeController.php



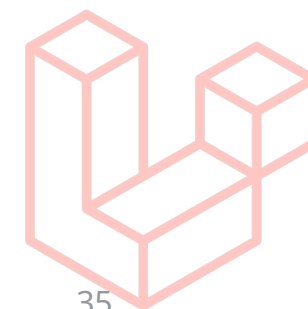
# MVC no Laravel : CONTROLLER

Padrão Model-View-Controller no Framework Laravel



- **Controllers:** Verifique o arquivo criado *HomeController.php*

```
app > Http > Controllers > 📄 HomeController.php > ...
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use Illuminate\Http\Request;
6
7  class HomeController extends Controller
8  {
9      //
10 }
11
```



# MVC no Laravel : CONTROLLER

Padrão Model-View-Controller no Framework Laravel



- **Controllers:** Criaremos o método *index()*

INTROLARAVEL

app

Console

Exceptions

Http

Controllers

Controller.php

HomeController.php

Middleware

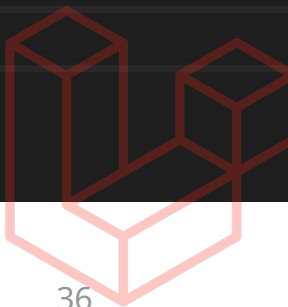
Kernel.php

Models

Providers

app > Http > Controllers > HomeController.php > PHP Intelephense >

```
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use Illuminate\Http\Request;
6
7  class HomeController extends Controller
8  {
9      public function index(){
10         echo "Olá Mundo!!!";
11         dd($this);
12     }
13 }
```



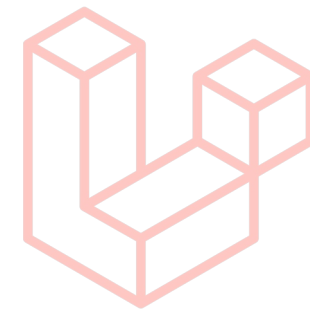
# MVC no Laravel : CONTROLLER

Padrão Model-View-Controller no Framework Laravel



- **Route:** Agora voltaremos a alterar a rota “ola” em *routes/web.php*.

```
routes >  web.php > ...  
1  <?php  
2  
3  use App\Http\Controllers\HomeController;  
4  use Illuminate\Support\Facades\Route;
```



- Indicamos com o **use** o carregamento da classe **HomeController**
- Passamos para o **Route::get** um array com a classe e o método que deve ser chamado: [ **NomeDaClasse::class**, ‘**nome\_do\_metodo**’ ]

```
20  
21 Route::get('/ola',[HomeController::class,'index']);
```

# MVC no Laravel : MODEL

Padrão Model-View-Controller no Framework Laravel

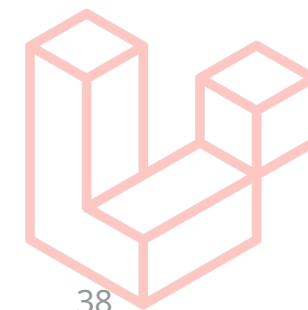
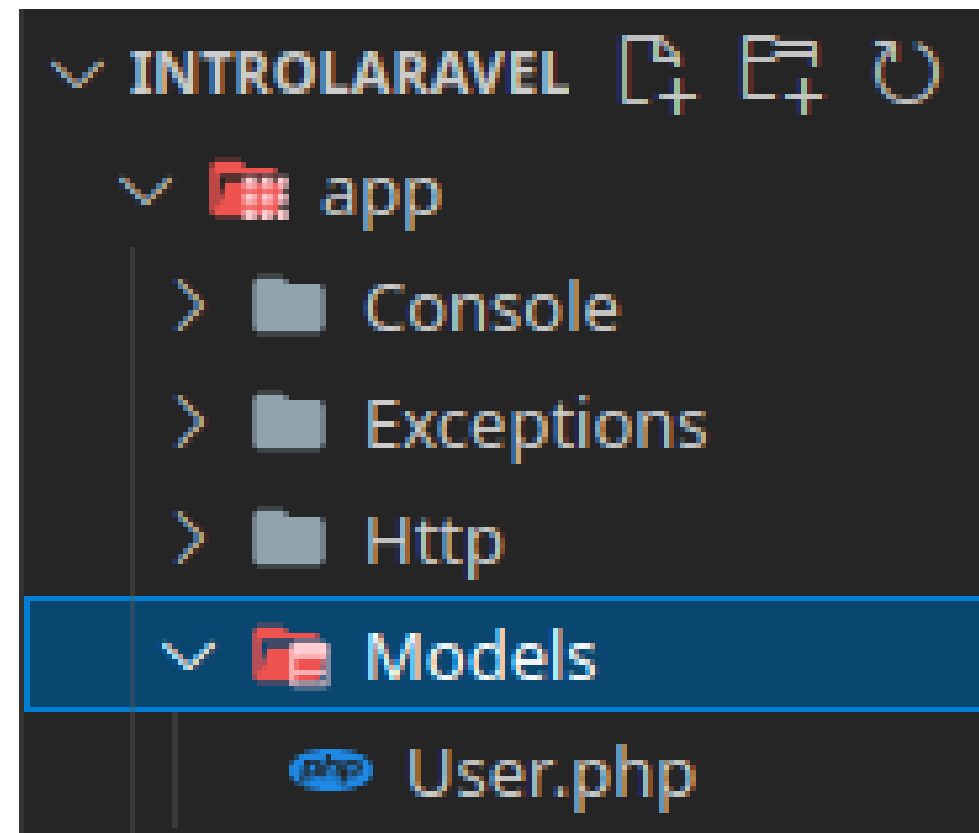


- **Model:**

- Extendem a classe **Model (Eloquent)**.
- Realizam o acesso ao banco através do **Eloquent**
- Padrão nome: **User** (Model) → **users** (Tabela)
- **snake\_case** → UserProfileImage → user\_profile\_images (migrations)
- Tabelas definidas por **Migrations**

-  **Eloquent** :

- Mapeamento Objeto Relacional do Laravel – POO → Tabelas (ER)
- Possui métodos para abstrair relações 1:n, 1:1 ou n:n
- Considera sempre como chave primária uma coluna **:id**





# MVC no Laravel : MODEL

Padrão Model-View-Controller no Framework Laravel



- **Model:** Exemplo de uma classe de Modelo

```
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7
8  class UserProfile extends Model
9  {
10     use HasFactory;
11     protected $fillable = ['phone', 'mobile_phone', 'about', 'social_networks'];
12     protected $table = 'user_profile';
13     public function user(){
14         return $this->belongsTo(related: User::class);
15     }
16 }
```

Colunas da tabela  
No Banco de Dados

```
8 class CreateTableUserProfile extends Migration
9 {
10     /**
11      * Run the migrations.
12      *
13      * @return void
14      */
15     public function up()
16     {
17         Schema::create( table: 'user_profile', function (Blueprint $table) {
18             $table->id();
19             $table->text( column: 'about')->nullable( value: 'true');
20             $table->text( column: 'social_networks')->nullable( value: true);
21             $table->string( column: 'phone');
22             $table->string( column: 'mobile_phone');
23             $table->timestamps();
24             $table->foreignIdFor( model: User::class)->constrained();
25         });
26     }
}
```

Migration da tabela  
*user\_profile* representada  
pela Model UserProfile

# MVC no Laravel : MODEL

Padrão Model-View-Controller no Framework Laravel



- **Model:**

- Para criar uma classe de modelo usaremos novamente o **artisan** com o comando **make**:

```
$> php artisan make:model Produto -mc
```

- No entanto, desta vez iremos criar também uma **Migration (-m)** e um controlador para a model **Produto (c)**.
- Verifique a pasta **app/Http/Controller** para a criação do arquivo **ProdutoController.php**.
- Verifique a pasta **database/migrations** para a criação do arquivo **create\_produtos\_table.php**.
- Com os arquivos criados o próximo passo agora é configurar os campos para a tabela **produtos** na migration **create\_produto\_table.php**!

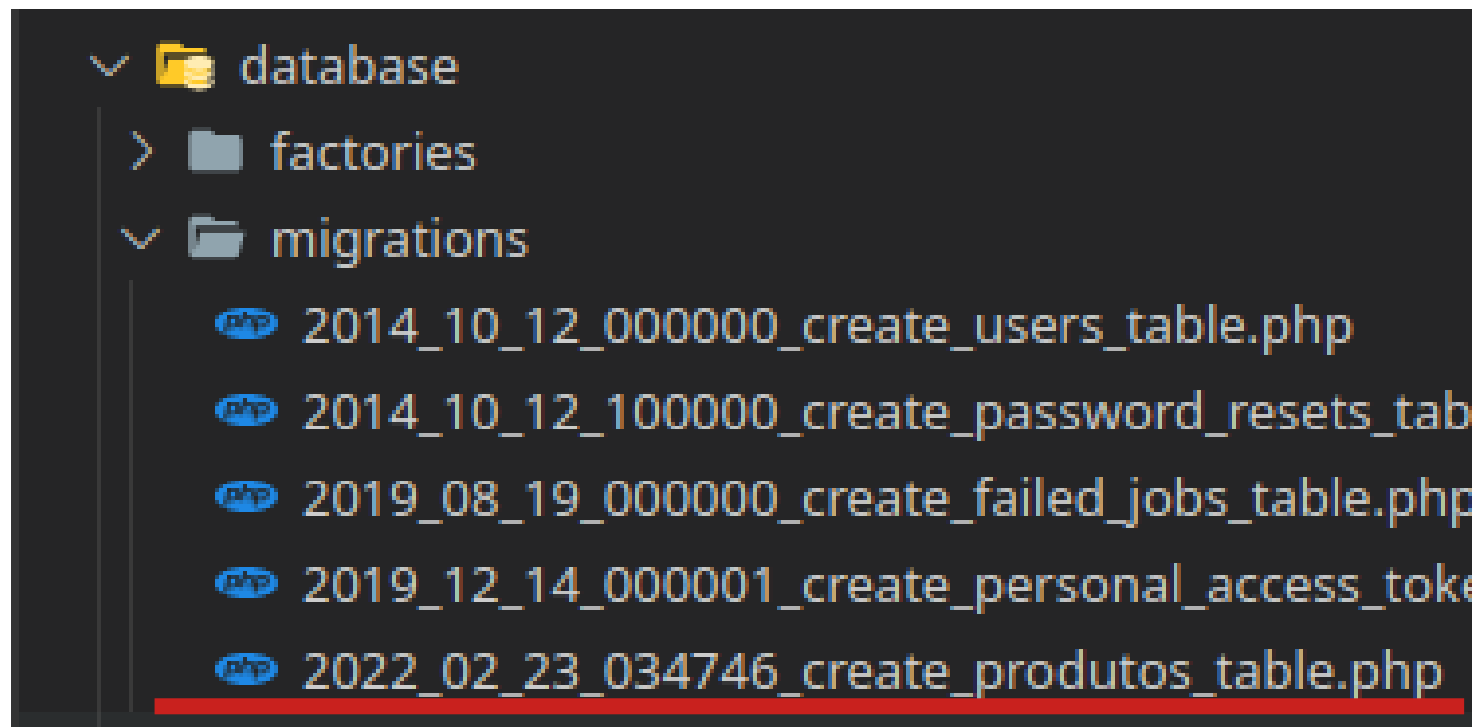
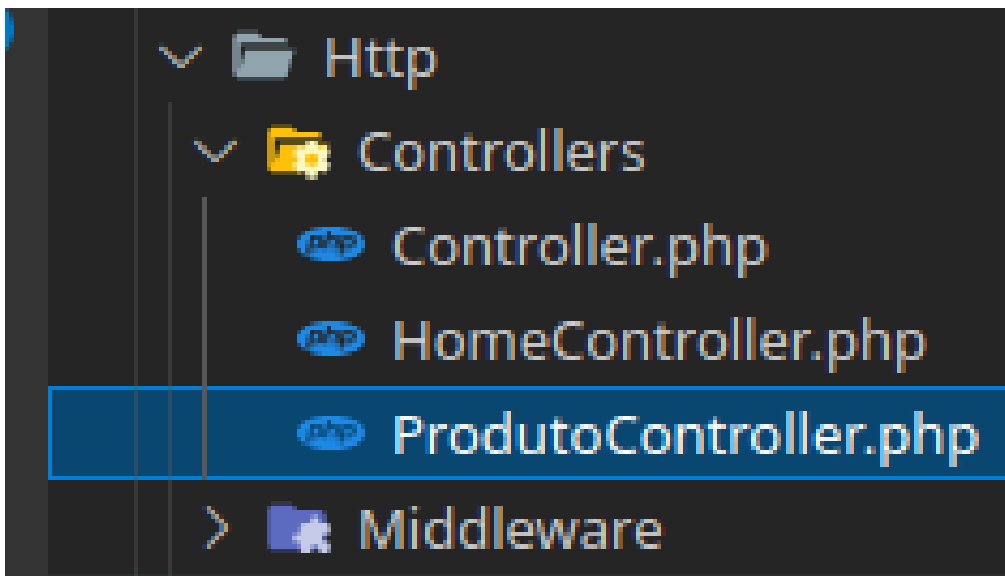
# MVC no Laravel : MODEL

Padrão Model-View-Controller no Framework Laravel



- **Model:** Criação do Model Produto com Migration e Controller no projeto

```
echoes@echoes-Inspiron-15-3567 > ...Projetos/tsi_daoo/introLaravel > <?PHP 7.4.3 > ↵  
$ php artisan make:model Produto -mc  
Model created successfully.  
Created Migration: 2022_02_23_034746_create_produtos_table  
Controller created successfully.
```



# MVC no Laravel : MODEL

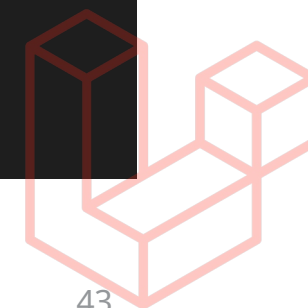
Padrão Model-View-Controller no Framework Laravel



- **Model:** Configurando os campos da migration *create\_produtos\_table* na classe *CreateProdutosTable*, edite o método *up()*;

```
14     public function up()
15     {
16         Schema::create('produtos', function (Blueprint $table) {
17             $table->id();
18             $table->text('nome');
19             $table->text('descricao');
20             $table->integer('qtd_estoque');
21             $table->float('preco');
22             $table->boolean('importado')->default(false);
23             $table->timestamps();
24         });
25     }
```

-  **Documentação de Migrations!**



# MVC no Laravel : MODEL

Padrão Model-View-Controller no Framework Laravel



- **Model:**
  - Antes de instalar as migrations vamos configurar o banco de dados
  - Edite o arquivo **.env** na raiz do projeto e procure as variáveis de banco de dados:

```
> vendor
.editorconfig
.env
.env.example
.gitattributes
.gitignore
```

```
10
11 DB_CONNECTION=mysql
12 DB_HOST=127.0.0.1
13 DB_PORT=3306
14 DB_DATABASE=intro_laravel
15 DB_USERNAME=root
16 DB_PASSWORD=r00t
```

# MVC no Laravel : MODEL

Padrão Model-View-Controller no Framework Laravel



- **Model:**

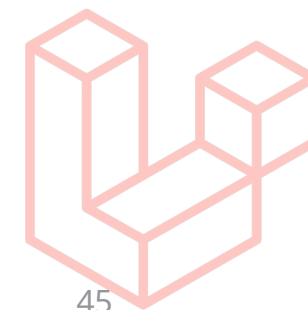
- Para criar as tabelas do banco de dados use o **artisan** com o comando **migrate** como mostrado a seguir:

```
$> php artisan migrate:install
```

- Se as migrations já foram executadas este comando pode ser ignorado e executar diretamente o comando **migrate:fresh**.
- Mesmo depois de configurado o arquivo **.env** apareça erro de acesso, tente fechar todos os terminais abertos e reiniciar o servidor artisan, mysql e apache.
- Em alguns casos as variáveis de ambiente não são totalmente atualizadas.
- Roda então o comando:

```
$> php artisan migrate:fresh
```

- Serão criadas várias tabelas por padrão no banco incluindo a tabela **produtos**.





# MVC no Laravel : MODEL

Padrão Model-View-Controller no Framework Laravel



- **Model:** Criando a tabelas com **migrate:fresh**

```
$ php artisan migrate:fresh
Dropped all tables successfully.
Migration table created successfully.
Migrating: 2014_10_12_000000_create_users_table
Migrated: 2014_10_12_000000_create_users_table (312.24ms)
Migrating: 2014_10_12_100000_create_password_resets_table
Migrated: 2014_10_12_100000_create_password_resets_table (144.80ms)
Migrating: 2019_08_19_000000_create_failed_jobs_table
Migrated: 2019_08_19_000000_create_failed_jobs_table (1,815.67ms)
Migrating: 2019_12_14_000001_create_personal_access_tokens_table
Migrated: 2019_12_14_000001_create_personal_access_tokens_table (308.10ms)
Migrating: 2022_02_23_034746_create_produtos_table
Migrated: 2022_02_23_034746_create_produtos_table (97.32ms)
```

- Após a criação da tabela insira os dados de produtos diretamente no banco com o sql disponibilizado no link a seguir: [!\[\]\(30a147af384f9f71632c2ff17bc706c8\_img.jpg\) \*\*SQL INSERT PRODUTOS\*\*](#)

# MVC no Laravel : MODEL

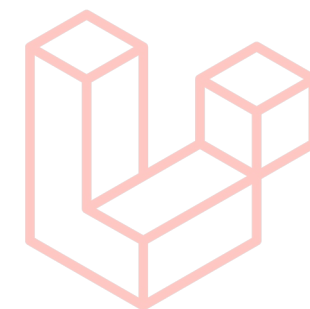
Padrão Model-View-Controller no Framework Laravel



- Usando a Model:

- Depois de importar os dados para a tabela **produtos** podemos acessar estes dados através do **Laravel** com as classes **Model** e **Controller** de **Produto** criadas.
- Para isso vamos primeiro criar a **rota** para acessar o controlador de produto no arquivo **/routes/web.php**:

```
3 use App\Http\Controllers\HomeController;  
4 use App\Http\Controllers\ProdutoController;  
5 use Illuminate\Support\Facades\Route;
```



```
22 Route::get('/ola', [HomeController::class, 'index']);  
23 Route::get('/produtos', [ProdutoController::class, 'index']);
```

# MVC no Laravel

## Padrão Model-View-Controller

- Usando a Model:

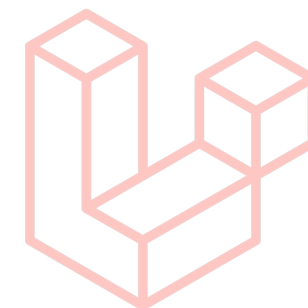
- Criaremos então o método index que será chamado na rota **'/produtos'**:
- Note o atributo **\$produto** como um objeto da classe Model **Produto()**, indicada pelo **use** (linha 5);
- O atributo **\$produto** é criado com um objeto da *model* **Produto** no construtor do *controller* **ProdutoController**

```
3 namespace App\Http\Controllers;
4
5 use App\Models\Produto;
6 use Illuminate\Http\Request;
7
8 class ProdutoController extends Controller
9 {
10     /**
11      * @var Produto
12      */
13     private $produto;
14
15     public function __construct()
16     {
17         $this->produto = new Produto();
18     }
19
20     public function index(){
21         // Mostrará o objeto retornado pelo all()
22         // dd($this->produto->all()); //debug do laravel
23
24         return response()->json($this->produto->all());
25     }
26 }
```



- **Usando a Model:**

- O método *all()* chamado pelo *index* retorna todos os dados da tabela *produtos*.
- Existem outros métodos do *Eloquent* para acesso aos dados:
  - *find(id)*: retorna o registro com o id passado como parâmetro



```
20 public function index(){
21     // Mostrará o objeto retornado pelo all()
22     // dd($this->produto->all()); //debug do laravel
23
24     return response()->json($this->produto->all());
25 }
```

# MVC no Laravel : MODEL

Padrão Model-View-Controller

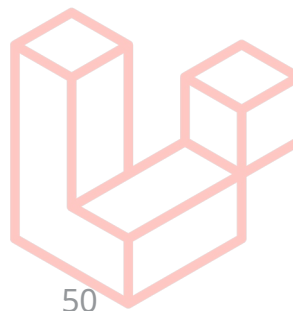


- **Testando a Model:** No navegador devem ser retornados 10 produtos.

```
localhost:8082/produtos

JSON Raw Data Headers
Save Copy Collapse All Expand All Filter JSON

▼ 0:
  id: 111
  nome: "Samsung A5 - 2017"
  descricao: "Samsung A5 2017 2GB Exynos 8Core"
  qtd_estoque: 2
  preco: 4500
  importado: 0
  created_at: null
  updated_at: null
▶ 1: {...}
▶ 2: {...}
▶ 3: {...}
▶ 4: {...}
```



# MVC no Laravel : VIEWS

Padrão Model-View-Controller no Framework Laravel

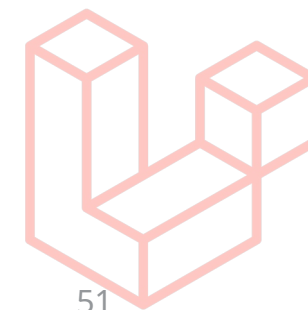
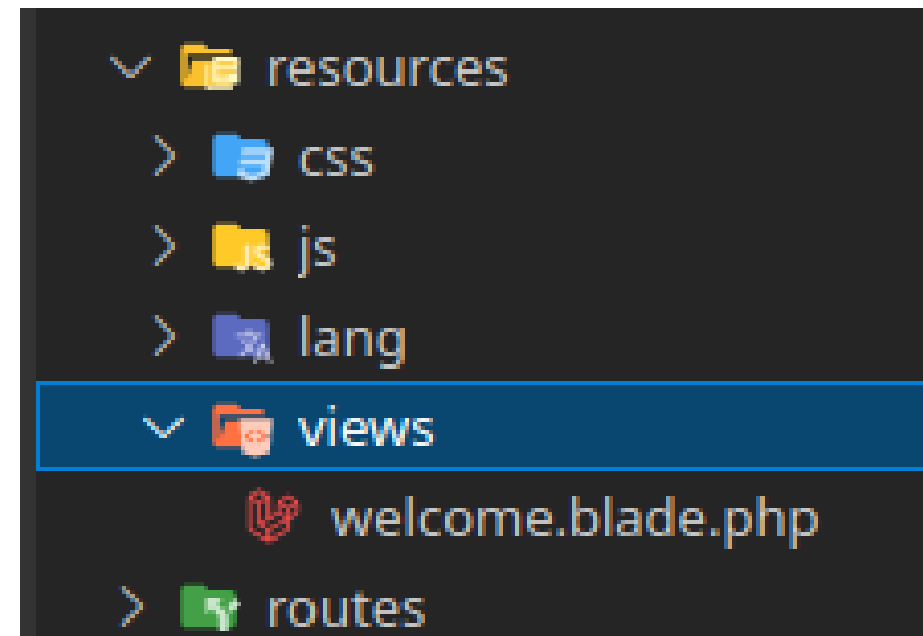


- **Views:**

- Localizados no diretório */resource/views/*
- Arquivos de template html (*.blade.php*).
- Motor de templates **Blade**
- A view é enviada por *rotas* ou pelos *controladores*.
- Usando a função `view('nome_template')` [*helpers*]

-  **Blade:**

- Aceita código PHP dentro do HTML
- Usa diretiva especiais para lógica e subdivisão de templates.
- Aceita outros frameworks JavaScripts (Vue, React, Angular etc...)



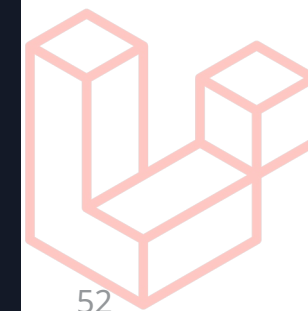
# MVC no Laravel : VIEW

Padrão Model-View-Controller



- **View:** Exemplo de um *template* blade.php para a view

```
1 @extends('layouts.app')
2 @section('home')
3     <div class='row justify-content-center'>
4         <div class="col-md-8 text-center">
5             <div class="card">
6                 <div class="card-header">
7                     <h1>Todos Page</h1>
8                 </div>
9                 <div class="card-body">
10                     <div class="panel panel-default text-center">
11                         {{-- <p><a href="{{ route('todo.create') }}">&plus;New Todo</a></p> --}}
12                         <p>
13                             <a href="{{ route('todo.create') }}">
14                                 <span class='fas fa-plus-circle' ></span>
15                                 New Todo
16                             </a>
17                         </p>
18                     </div>
19                     @if (session()->has('alert_type') && session()->has('alert_message'))
20                         <x-alert type="{{ session()->get('alert_type') }}"
21                             message="{{ session()->get('alert_message') }}" />
22                     @else
23                         <x-alert />
24                     @endif
25                 </div>
18             </div>
19         </div>
20     </div>
21 @endsection
```





# MVC no Laravel : VIEWS

Padrão Model-View-Controller no Framework Laravel



- **Views:**

- Como pode ser notado na imagem do slide anterior o template blade possui **diretivas** (@) que funcionam como estruturas lógicas:

- Ex: **@if** - **@else** **@endif**, **@foreach** - **@endforeach**

- Também é possível receber variáveis e tratá-las dentro de **chaves duplas**:

- Ex: imprimir o valor da variável **\$text** dentro do parágrafo **<p>**

```
<p>{{ $text → value() }} </p>
```

- Outra característica do Blade é a possibilidade de dividir ou reaproveitar partes de templates como cabeçalhos e rodapés com as diretivas **@extends** e **@section**

- Basicamente **@extends** aponta para uma página template **layout/app.blade.php** onde será inserida a sessão definida por **@section**

# MVC no Laravel : VIEWS

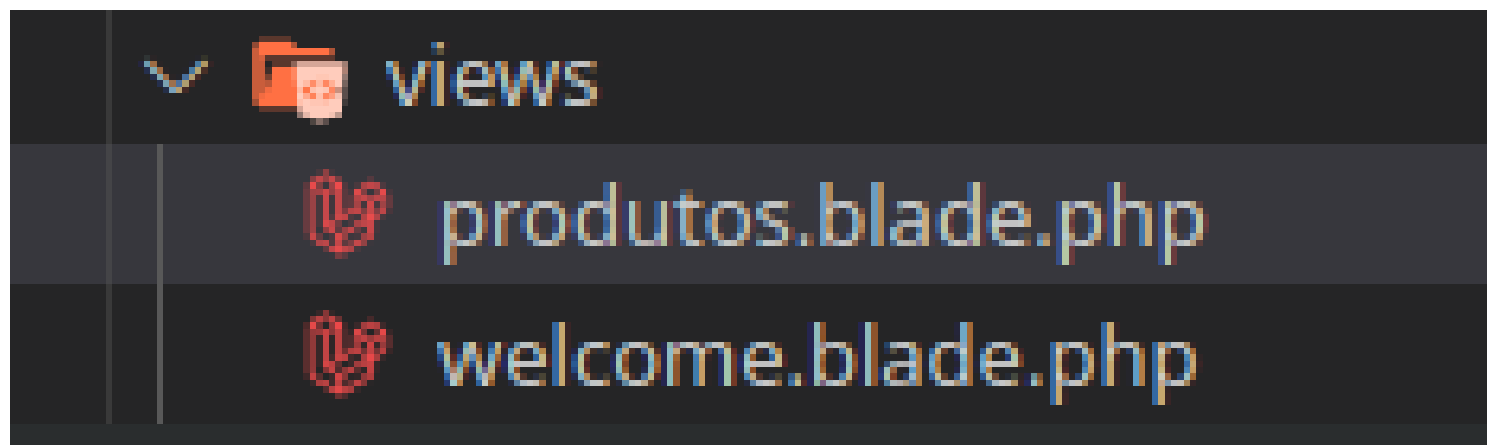
Padrão Model-View-Controller no Framework Laravel



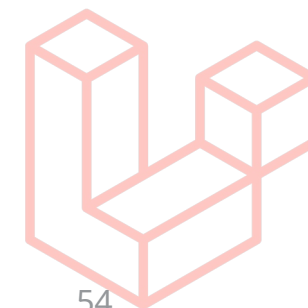
- **Views:**

- Vamos criar um *template* para a página que listará os produtos.
- Portanto criaremos o arquivo ***produtos.blade.php*** na pasta ***/resource/views***.

```
$> php artisan make:view produtos
```



- Depois de criado o arquivo copie o *html* do exemplo neste  *Link*;



# MVC no Laravel : VIEWS

Padrão Model-View-Controller no Framework Laravel



- **Views:**

- Agora vamos alterar a classe **ProdutoController** para chamar o *template* criado ao em vez de apenas retornar um json.

```
20 public function index(){
21     // Mostrará o objeto retornado pelo all()
22     // dd($this->produto->all()); //debug do laravel
23
24     //retorna um resposta Http em formato JSON
25     // return response()->json($this->produto->all());
26
27     //retorna uma view e passa os dados para o template
28     return view('produtos',['produtos'=>$this->produto->all()]);
29 }
```

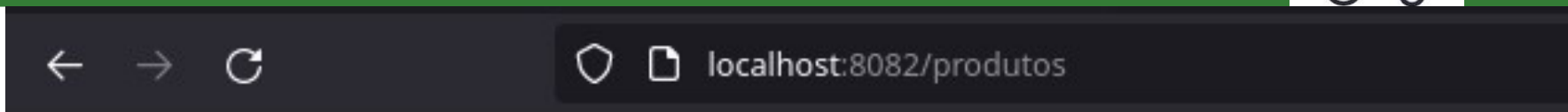
# MVC no Laravel : VIEWS

Padrão Model-View-Controller no Framework Laravel



- **Views:**

- Ao acessar a página na rota “/produtos” será carregado o *template* mostrando os dados da tabela *produtos*.



## Produtos

| Id  | Nome                       | qtd_estoque | preco | Importado |
|-----|----------------------------|-------------|-------|-----------|
| 111 | Samsung A5 - 2017          | 2           | 4500  | Não       |
| 112 | Notebook DELL Inspiron 15  | 300         | 8500  | Não       |
| 113 | Notebook Samsung Gamer     | 150         | 17500 | Não       |
| 114 | SSD 4TB                    | 200         | 5750  | Não       |
| 115 | SSD 2TB                    | 150         | 3750  | Não       |
| 121 | SSD 4TB                    | 50          | 4150  | Não       |
| 122 | GAINWARD PHOENIX RTX3080ti | 30          | 14150 | Não       |
| 123 | GAINWARD PHOENIX RTX3070   | 60          | 7399  | Não       |
| 124 | ECHO DOT ALEXA             | 1000        | 200   | Não       |
| 125 | Monitor Asus BK 35"        | 500         | 9990  | Não       |

# MVC no Laravel : VIEWS

Padrão Model-View-Controller no Framework Laravel



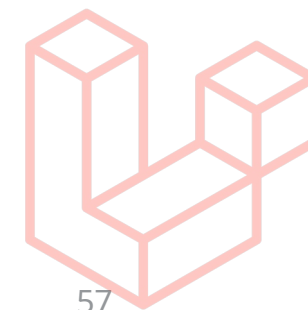
- **Views:** Note que a **chave** do array associativo passado para a função view chamada no controlador, será a variável a ser recuperada no template blade.

```
28 return view('produtos', ['produtos' => $this->produto->all()]);
```

app/Http/ProdutoController.php

resources/views/produtos.blade.php

```
<tbody>
  @foreach($produtos as $produto)
    <tr>
      <td>{{ $produto->id }}</td>
      <td>{{ $produto->nome }}</td>
      <td>{{ $produto->qtd_estoque }}</td>
      <td>{{ $produto->preco }}</td>
      <td>{{ ($produto->Importado) ? 'Sim' : 'Não' }}</td>
    </tr>
  @endforeach
</tbody>
```





- **Tarefas:**

- 1) Criar a página *single* de produtos:**

- Considerando que podemos passar um ID na **rota** usando: **Route::get('/nome\_da\_rota/{id}', [classe, 'metodo'])**.
- E que esse parametro poderá ser recebido no método da classe.
- Considerando também o método **find()** da Model, além do método **all()** visto nos exemplos.
- Implemente o método **show** na classe **ProdutoController** para mostrar os dados de um único *produto* ao receber o seu **ID**.
- Implemente a página única de cada produto conforme o exemplo nos próximos slides.

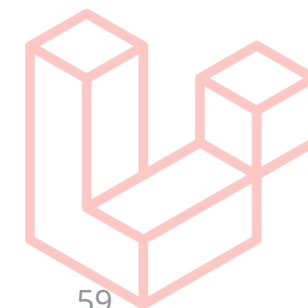


localhost:8082/produtos

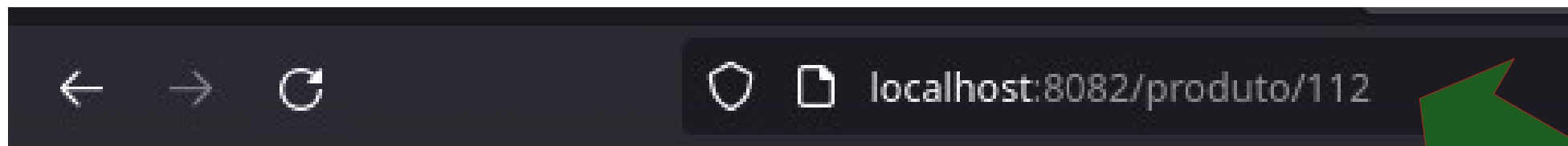
## Produtos



| Id                  | Nome                      | qtd_estoque | preço |
|---------------------|---------------------------|-------------|-------|
| <a href="#">111</a> | Samsumg A5 - 2017         | 2           | 45    |
| <a href="#">112</a> | Notebook DELL Inspiron 15 | 300         | 85    |
| <a href="#">113</a> | Notebook Samsumg Gamer    | 150         | 17    |
| <a href="#">114</a> | SSD 4TB                   | 200         | 57    |
| <a href="#">115</a> | SSD 8TB                   | 150         | 85    |







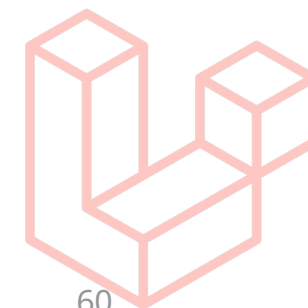
## Notebook DELL Inspiron 15

I5 7600HQ 8GBMen GTX1030m SSD 1TB

- Quantidade: 300
- Preço: 8500
- Importado: Não

[voltar](#)

Deve voltar para a  
lista de Produtos





## 2) Criar as classes *Model* e *Controller* de três tabelas do projeto/TCC:

- Utilizando o *artisan*:
  - Crie a *Model*;
  - Crie o *Controller*;
  - Crie a *Migration* e edite as colunas;
- Após execute o comando do *artisan* para criação as tabelas no banco de dados!
- Com as tabelas criadas, insira valores através do gerenciador do seu banco de dados, ***phpmyadmin***, ***MySQL Workbench*** ou ***pgAdmin***.
- Laravel é independente de SGBD!!





### 3) Criar a camada de view para as três tabelas do seu projeto/TCC:

- Utilize o *blade* para a criação dos templates de *Lista* e *Single* de cada tabela.
- Pode utilizar os templates de exemplo, pois o foco não é no template mas a funcionalidade e o uso do *framework*.
- Crie um repositório privado no **github** para o seu projeto laravel e adicione o usuário do professor como colaborador.
- Crie uma branch para esta atividade em seu repositório, pode usar o nome ***aula\_05***.
- Envie o link do repositório remoto na atividade referente a esta aula no moodle.

**OBS:** Salve na pasta ***database/dump*** um dump do seu banco de dados com os dados, não apenas o sql de criação.